

Guia do Desenvolvedor — Addon Sankhya <-> FastChannel

Documentacao de handoff para continuidade, alteracoes, manutencao e suporte do addon.

Gerada em 2026-07-01 a partir da leitura direta do codigo-fonte (branch merge/unify-fastchannel, v1.2.91).

Complementa o CHANGELOG.md (historico detalhado v1.2.x) e o OPERATIONS_RUNBOOK.md (operacao/troubleshooting).

Arquitetura & Visão Geral — Addon Sankhya ↔ FastChannel

Módulo de integração e-commerce (FastChannel / Data Lake) empacotado como Add-on Sankhya (Add-on Studio 2.0.0), rodando dentro do WildFly do Sankhya ERP. AppKey `b262dd0f-3ca5-4219-8053-7a492f792590`, `platform-min-version 4.28`, vendedor BEL DISTRIBUIDOR DE LUBRIFICANTES LTDA (`addon-fastchannel.ear/extension.xml:1-16`). A versão do `extension.xml` (1.2.53) está defasada em relação ao `CHANGELOG.md` (v1.2.91) — o versionamento real de código está no CHANGELOG, não no descritor EAR.

Código Java todo sob `model/src/main/java/br/com/bellube/fastchannel/`. Este documento reflete a leitura direta do código-fonte, do `CHANGELOG.md` (histórico v1.2.x) e do `README.md`.

1. Módulos (`model/` , `vc/` , `addon-fastchannel.ear/`)

O projeto é um multi-módulo Gradle (`settings.gradle` , `build.gradle` de 55 KB na raiz) que gera um EAR de add-on Sankhya.

Módulo	Papel	Conteúdo principal
<code>model/</code>	Camada de modelo e lógica de negócio (EJB/JAPE). É onde vive todo o código Java (<code>model/src/main/java/br/com/bellube/fastchannel/</code>): listeners JAPE, jobs, services, HTTP clients, DTOs, ciclo de vida. Empacotado como o JAR de modelo do add-on.	~90 classes Java.
<code>vc/</code>	Camada "View/Controller" — o webapp (WAR). Contém os recursos HTML5 do frontend (<code>vc/src/main/resources</code> → <code>html5/fastchannel/*.html</code> , <code>html5/FastchannelMonitor/</code>), <code>META-INF/addon-fastchannel-dashboards</code> , <code>WEB-INF/customization</code> e <code>WEB-INF/resources</code> . Serve o Dashboard, Config, Pedidos, Fila, Logs (ver §5 abaixo e <code>README.md:29-36</code>).	<code>build.gradle</code> próprio; recursos estáticos e customizações de tela.
<code>addon-fastchannel.ear/</code>	Estrutura EAR de deployment do add-on. Contém <code>extension.xml</code> (descritor do add-on: id <code>addon-fastchannel</code> , appkey, versão, <code>platform-min-version</code>) e <code>META-INF/</code> . É o artefato final copiado para <code>X:\Wildfly_Clean\wildfly_producao\standalone\deployments\</code> (<code>README.md:57-62</code>).	<code>extension.xml</code> , <code>META-INF/</code> .

Datasource: acesso JNDI direto ao `java:/MGEDS` (datasource do MGE no WildFly) via `DBUtil / DbColumnSupport` . O add-on **não** usa persistência JPA própria — grava direto nas tabelas `AD_FC*` e lê as tabelas nativas do Sankhya (`README.md:38-48`).

Nota de empacotamento (do CHANGELOG / memória do projeto): o `build.gradle` tem tasks críticas para o add-on funcionar em runtime — `stripJarsFromWar` (remove `libs-master.jar` e `mge-modelcore-master.jar` do WAR para evitar o `JapeSessionLifecycleImpl cannot be cast` , causado por classloader isolation) e o `settings.gradle / rootProject.name` em kebab-case lowercase (senão os descritores `META-INF` ficam sem prefixo e o TinyEJB não carrega). Isso não é "arquitetura de aplicação" mas é pré-requisito de deploy — quem for manter deve preservar essas tasks.

2. Mapa dos pacotes (`br.com.bellube.fastchannel.*`)

Pacote	Responsabilidade	Classes-chave
action	Ações disparadas pela UI (botões do dashboard/telas) — operações manuais/administrativas.	<code>ImportarPedidosAction</code> , <code>ProcessarFilaAction</code> , <code>SincronizarProdutosAction</code> , <code>ReenviarStatusPedidoAction</code> , <code>ReprocessarItemAction</code> , <code>ConsultarPedidoFCAction</code> , <code>TestarConexaoAction</code> , <code>TestarEstrategiasAction</code> , <code>LimparLogsAction</code> , <code>LimparErrosFataisAction</code>
auth	Autenticação/token OAuth com a API FastChannel.	<code>FastchannelTokenManager</code>
config	Configuração da integração (lida de <code>AD_FCCONFIG</code> , cache 5 min) e constantes.	<code>FastchannelConfig</code> , <code>FastchannelConstants</code> , <code>SourceConfig</code>
dto	Objetos de transferência de/para a API FastChannel e para a fila.	<code>OrderDTO</code> , <code>OrderItemDTO</code> , <code>OrderCustomerDTO</code> , <code>OrderAddressDTO</code> , <code>OrderInvoiceDTO</code> , <code>OrderPaymentDetailsDTO</code> , <code>OrderTrackingDTO</code> , <code>OrderStatusDTO</code> , <code>PriceDTO</code> , <code>PriceBatchDTO</code> , <code>PriceBatchItemDTO</code> , <code>StockDTO</code> , <code>QueueItemDTO</code>
exception	Hierarquia de exceções tipadas para tratar retry vs. falha fatal.	<code>FastchannelException</code> (base), <code>FastchannelAuthException</code> , <code>FastchannelRateLimitException</code> , <code>FastchannelTransientException</code> , <code>FastchannelFatalException</code>
http	Cientes HTTP das APIs REST da FastChannel (Order/Price/Stock Management).	<code>FastchannelHttpClient</code> (base), <code>FastchannelOrdersClient</code> , <code>FastchannelPriceClient</code> , <code>FastchannelStockClient</code>
installation	Ciclo de vida do add-on: bootstrap do scheduler, provisionamento, controller de install/uninstall/verify.	<code>FastchannellifecycleListener</code> , <code>FastchannelAutoProvisioning</code> , <code>LifecycleController</code>
job	Jobs agendados (implementam <code>EventoProgramavelJava</code> do Sankhya).	<code>OrderImportJob</code> , <code>OrderStatusSyncJob</code> , <code>OutboxProcessorJob</code> , <code>PriceFullSyncJob</code> , <code>StockFullSyncJob</code> , <code>AutoPriceChangesSweepJob</code> , <code>FCVlrNotaRepairJob</code>
listener	Listeners JAPE (<code>@Listener</code>) que reagem a mudanças em tabelas Sankhya e enfileiram no outbox.	<code>EstoqueListener</code> , <code>PrecoListener</code> , <code>ProdutoListener</code> , <code>DescontoPromocionalListener</code> , <code>NotaFiscalListener</code> , <code>TrackingListener</code>
service	Lógica de negócio principal (orquestração de importação de pedido, preço, estoque, De-Para, fila, logs, notificação).	<code>OrderService</code> , <code>OrderXmlBuilder</code> , <code>PriceService</code> , <code>PriceResolver</code> , <code>PriceTableResolver</code> , <code>PriceBatchResolver</code> , <code>StockResolver</code> , <code>QueueService</code> , <code>DeparaService</code> , <code>LogService</code> , <code>NotificationService</code> , <code>FastchannelHeaderMappingService</code> , <code>PartnerLookupUtil</code> , <code>SankhyaServiceInvoker</code>
service/strategy	Strategy + fallback para criar o pedido no Sankhya (várias abordagens de inclusão de nota).	<code>OrderCreationStrategy</code> (interface), <code>OrderCreationOrchestrator</code> , <code>ServiceInvokerStrategy</code> , <code>InternalApiStrategy</code> , <code>HttpServiceStrategy</code>
service/auth , service/nativeapi	Sub-pacotes: gestão de sessão Sankhya e invocação de serviços nativos.	<code>SankhyaAuthManager</code> , <code>SankhyaNativeServiceCaller</code>
util	Utilitários de BD, filtro de produto, parâmetros, sanitização de log.	<code>DBUtil</code> , <code>DbColumnSupport</code> , <code>FastchannelProductFilter</code> , <code>FastchannelParameters</code> , <code>LogSanitizer</code>
web	Servlets/serviços HTTP para o frontend HTML5 e filtros.	<code>FastchannelDirectServlet</code> (roteador /fc-direct), <code>FCDashboardService</code> , <code>FCConfigService</code> , <code>FCPedidosService</code> , <code>FCDeparaService</code> , <code>FCEstoqueService</code> , <code>FCPrecosService</code> , <code>FCAdminService</code> , <code>FCLogsService</code> , <code>FcSessionBridgeFilter</code> , <code>FcRequestDebugFilter</code>

3. Ciclo de vida do add-on

Dois pontos de entrada trabalham juntos: o `LifecycleController` (chamado pelo hub Sankhya via HTTP no install/uninstall/verify) e o `FastchannellifecycleListener` (`ServletContextListener` do WAR, para shutdown limpo em redeploy). Ambos convergem para `FastchannelAutoProvisioning`, que gerencia o scheduler interno.

3.1 LifecycleController (installation/LifecycleController.java)

`ServiceWrapper` / `IServiceWrapperProvider` cujos métodos públicos são invocados diretamente pelo ServiceProvider do add-on (`handle()` é dispatcher vazio):

- `install(ctx)` → `FastchannelAutoProvisioning.ensureStarted(appkey, codmodulo)` → responde `INSTALLED`.
- `verify(ctx)` → também chama `ensureStarted(...)` (idempotente) → responde `OK`.
- `uninstall(ctx)` → `FastchannelAutoProvisioning.stopAll(...)` → responde `UNINSTALLED`.
- `deleteWithFallback(ctx)` / `safeDelete(ctx)` → respostas locais `DELETED_LOCAL` (não chamam o servidor remoto).

3.2 FastchannellifecycleListener (installation/FastchannellifecycleListener.java)

`ServletContextListener` que resolve o problema de **classloaders zumbis em redeploy** ([CRIT 2026-04-27]/[FIX 2026-04-28]):

- `contextInitialized`: varre **todas** as threads da JVM e interrompe as chamadas `fastchannel-auto-*` cujo `contextClassLoader` seja diferente do atual (remanescentes de versões antigas v8→v10→v11 que rodavam em paralelo, disparando jobs 2× por ciclo). Estratégia segura: só `interrupt()` (`interruptZombieFastchannelThreads()`).
- `contextDestroyed`: chama `FastchannelAutoProvisioning.stopAll(null, null)` para desligar o scheduler interno antes do classloader ser liberado.

3.3 FastchannelAutoProvisioning (installation/FastchannelAutoProvisioning.java)

Coração do provisionamento. `ensureStarted(appKey, explicitCodModulo)` :

1. **Tenta o scheduler nativo do Sankhya** (`tryStartNativeScheduledActions`) : resolve `CODMODULO` (por `appKey` via `INFORMATION_SCHEMA` ou explícito), consulta `br.com.sankhya.acaoagendada.ScheduledActionsUtils.moduleHasActions(...)` por reflexão e, se houver Ações Agendadas ativas, faz `refreshJob / startJob` em cada `NUAAG` .

2. **Fallback interno** (`startInternalFallback`) : se não houver ações nativas, ativa um `ScheduledExecutorService` idempotente (guardas `INTERNAL_STARTED` , `RUN_GUARD` , `LAST_ACTUAL_RUN_MS`).

3. **One-shot** `runOneShotPurgeAdNumFastDups()` : purga duplicatas `AD_NUMFAST` remanescentes lendo `AD_FCDUPPURGE` (skip silencioso se a tabela não existir) e aplicando `UPDATE TGFCAB SET AD_NUMFAST = NULL` via pool JDBC do DataSource (herda as SET options corretas, contornando o bug da trigger `TRG_INC_UPD_DLT_TGFFIN_SSPMB`). Fail-open.

Pool de threads: `Executors.newScheduledThreadPool(10, factory)` — aumentado de 5→10 em [FIX 2026-04-24] porque 5 threads e 6+ tasks causavam *starvation* (os jobs pesados `price-full / stock-full` nunca rodavam). Threads são daemon, nomeadas `fastchannel-auto-<id>` .

Jobs agendados (via `schedule(...)` — período fixo — ou `scheduleDynamic(...)` — intervalo lido da config a cada tick):

Nome (chave)	Método/Job	Intervalo (default)	System property override	Agendamento
depara-sync	<code>DeparaService.getInstance().syncProductDeparaFromRefform()</code>	24 h	<code>fc.auto.depara.hours</code>	<code>schedule</code>
order-import	<code>new OrderImportJob().executeScheduler()</code>	<code>cfg.getIntervalOrders()</code> min (de <code>AD_FCCONFIG</code>)	— (config)	<code>scheduleDynamic</code> , initialDelay 30 s
outbox	<code>new OutboxProcessorJob().executeScheduler()</code>	<code>cfg.getIntervalQueue()</code> min (de <code>AD_FCCONFIG</code>)	— (config)	<code>scheduleDynamic</code> , initialDelay 60 s
status-sync	<code>new OrderStatusSyncJob().executeScheduler()</code>	3 min	<code>fc.auto.status.minutes</code>	<code>schedule</code>
price-full	<code>new PriceFullSyncJob().executeScheduler()</code>	6 h	<code>fc.auto.price.hours</code>	<code>schedule</code>
stock-full	<code>new StockFullSyncJob().executeScheduler()</code>	6 h	<code>fc.auto.stock.hours</code>	<code>schedule</code>
auto-sweep-prices	<code>new AutoPriceChangesSweepJob().run()</code>	10 min	<code>fc.auto.sweep.minutes</code>	<code>schedule</code>

`AutoPriceChangesSweepJob` também usa `-Dfc.auto.sweep.expiredDays` (janela de detecção de promos expiradas; 2→30 dias em v1.2.91). `FCVlrNotaRepairJob` existe no pacote `job` como rede de segurança de `VLRNOTA` , mas **não está na lista de agendamento** deste `startInternalFallback` (não localizado no scheduler interno — ver lacunas).

`schedule(...)` — **armadilha de unidade** ([FIX 2026-05-15]): o `initialDelay` era calculado em segundos mas passado com a `TimeUnit` original (HOURS/MINUTES), fazendo o Java interpretar "120" como 120 **horas** = 5 dias (o `stock-full` nunca rodava). Correção: tudo convertido para `SECONDS` (`initialDelaySecs = clamp(20..120, unit.toSeconds(period))` , `periodSecs = unit.toSeconds(period)`). Quem alterar intervalos deve manter tudo em segundos internamente.

`scheduleDynamic(...)` : roda a cada 60 s mas só executa a task quando `now - lastRun >= intervalMsSupplier(cfg)` — permite mudar o intervalo em `AD_FCCONFIG` sem restart. Ambos os schedulers pulam a execução se `!cfg.isAtivo()` e usam guarda `AtomicBoolean` (`compareAndSet`) para não sobrepor execuções.

`runInJapeSession(name, task)` ([CRIT-4]) — **crítico e obrigatório** para qualquer job novo: envolve a task em `JapeSession.open()` / `close()` . Sem isso, a thread daemon do `ScheduledExecutorService` não tem contexto EJB → `EntityFacadeFactory.getCoreFacade()` falha ("Erro ao inicializar datasource para provider mge-core"), `AD_FCMAP` (`upsertOrderMapping`) não é gravado (Dashboard mostra "Importados Hoje: 0"), e `CACSP.incluirNota` grava com `CODUSU=0` . É **fail-open**: se `JapeSession.open()` falhar (classloader), roda a task mesmo assim para não travar a importação.

`stopInternalFallback()` ([CRIT 2026-04-27]): `shutdownNow()` + `awaitTermination(15s)` — necessário porque as threads podem estar bloqueadas em I/O (HTTP FC / JDBC) e o classloader antigo não seria liberado, criando pools paralelos no redeploy.

4. Listeners JAPE registrados (listener/*.java)

Todos estendem `PersistenceEventAdapter` e são anotados com `@Listener(instanceNames = {...})` . Padrão comum: no `afterInsert/afterUpdate/afterDelete` , checam `FastchannelConfig.isAtivo()` , resolvem SKU via `DeparaService` e enfileiram no outbox (`AD_FCQUEUE`) via `QueueService` (nunca chamam a API FC direto — o `OutboxProcessorJob` envia).

Classe	instanceNames (entidade JAPE)	Tabela Sankhya	Eventos	O que faz
EstoqueListener	Estoque	TGFEST	insert/update/delete	Filtra por local/empresa configurados (<code>isConfiguredLocalEmpresa</code>); resolve SKU (<code>getSkuForStock</code>); <code>queueService.enqueueStock(codProd, sku, estoque, codEmp, codLocal)</code> . Transaccional Outbox.
PrecoListener	Excecao , ExcecaoPreco	TGFEXC (Exceção de Preço)	insert/update/delete	Enfileira PRECO. [FIX 2026-06-01 v1.2.90] : o nome correto da instância é <code>Excecao</code> — antes era só <code>ExcecaoPreco</code> (inexistente no dicionário), então o listener nunca disparava e alterações de preço só chegavam pelo <code>auto-sweep-prices</code> (lag ~15 min). Filtra por tabelas de preço elegíveis (<code>PriceTableResolver</code>).
ProdutoListener	Produto	TGFPRO	insert/update/delete	Atualiza o De-Para (<code>setMapping(TIPO_PRODUTO, codProd, referencia)</code>) usando <code>REFERENCIA</code> como SKU; se produto inativo (<code>ATIVO != 'S'</code>) enfileira estoque zero.
DescontoPromocionalListener	Desconto , DescontoPorQuantidade	TGFDES (promoção) e TGFDPPQ (faixas de qtd / preço escalonado)	insert/update/delete	Resolve <code>CODPROD</code> , enfileira PRECO (<code>enqueuePrice</code>) para re-sync de batches escalonados. Introduzido 2026-04-24 (antes, preços escalonados só atualizavam por sync manual).
NotaFiscalListener	CabecalhoNota	TGFCAB (TGFCNOT)	update	Captura faturamento: resolve <code>orderId</code> FC por <code>NUNOTA</code> ; se <code>isSyncStatusEnabled</code> , processa criação de invoice (<code>CHAVENFE</code> presente → envia a NF para FC) e mudança de status. [FIX 2026-05-11] : <code>repairVlrNotaAfterConfirmation</code> re-aplica a fórmula correta de <code>VLRNOTA</code> (a proc nativa <code>STP_CONFIRMNOTA2</code> recalcula sem <code>VLRDESC</code> , quebrando cupom FC). Usa cache <code>INVOICE_TERMINAL_STATES</code> (máx 5000) para evitar spam de tentativas a cada <code>afterUpdate</code> trivial.
TrackingListener	Volume	TGFBVOL	insert/update	Captura expedição: lê <code>CODRASTREIO</code> (campo opcional), resolve <code>orderId</code> por <code>NUNOTA</code> e envia tracking (código de rastreamento/transportadora) ao FC via <code>FastchannelOrdersClient</code> . Só atua se <code>isSyncStatusEnabled</code> .

5. Fluxo de dados de alto nível Sankhya ↔ FastChannel

APIs FastChannel (`config/FastchannelConstants.java` , base `https://api.commerce.fastchannel.com`):

- Pedidos: `.../order-management/v1` (`ORDER_API_BASE`)
- Estoque: `.../stock-management/v1` (`STOCK_API_BASE`)
- Preço: `.../price-management/v1` (`PRICE_API_BASE`)

Padrão central: Transactional Outbox. Listeners JAPE e jobs de varredura **não** chamam a API FC diretamente — eles gravam na fila `AD_FCQUEUE` (`QueueService` , JDBC direto via `java:/MGEDS`). O `OutboxProcessorJob` (`outbox` , ~1 min) consome a fila e chama a API FC. Jobs "full sync" (`price-full` , `stock-full`) são a rede de segurança periódica; `auto-sweep-prices` capta mudanças que os listeners não pegam.

Estoque (Sankhya → FC)

`TGFEST` muda → `EstoqueListener` → `QueueService.enqueueStock` grava `AD_FCQUEUE` (TIPO ESTOQUE) → `OutboxProcessorJob` → `StockResolver` / `FastchannelStockClient` → PUT Stock Management. `StockFullSyncJob` reconcilia periodicamente.

Preço (Sankhya → FC)

`TGFEXC` / `TGFDES` / `TGFDPPQ` mudam → `PrecoListener` / `DescontoPromocionalListener` → `enqueuePrice` → `AD_FCQUEUE` (TIPO PRECO) → `OutboxProcessorJob` → `PriceService` / `PriceResolver` (preço via `SNK_GET_PRECO` , convertido para centavos) + `PriceBatchResolver` (faixas escalonadas) → `FastchannelPriceClient` (`price-management/v1` , incl. POST `.../prices/{sku}/batches`). `PriceFullSyncJob` (safety net) + `AutoPriceChangesSweepJob` (varre `TGFEXC.DHALTREG` e promos `TGFDES.DTFINAL` expiradas nos últimos 30 dias).

Pedido (FC → Sankhya)

`OrderImportJob` (`order-import` , intervalo `AD_FCCONFIG.INTERVAL_ORDERS`) → `OrderService.importarPedidos` busca pedidos pendentes (`FastchannelOrdersClient` , `order-management/v1`) → localiza/cria parceiro (`TGFPAR` , via `PartnerLookupUtil`) → cria cabeçalho (`TGFCAB`) e itens (`TGFITE`) via `OrderCreationOrchestrator` (estratégias em cascata com fallback) → registra em `AD_FCPEDIDO` / `AD_FCMAP` (`AD_NUMFAST` no `TGFCAB` amarra o pedido FC ao Sankhya).

Estratégias de criação (`OrderCreationOrchestrator` , ordem trocada em v1.2.77):

1. `ServiceInvokerStrategy` (**primária desde v1.2.77**) — invocação nativa `CACSP.incluirNota` com XML (`OrderXmlBuilder`), envolvida em

JapeSession.open() ([FIX 2026-04-28]).

- 2. InternalApiStrategy — API interna Sankhya/JapeWrapper (era a primária; conflitava com o recálculo nativo MGECom/ STP_CONFIRMNOTA2 , exigindo 4 camadas de "repair").
- 3. HttpServiceStrategy — último recurso: login/logout HTTP para obter JSESSIONID e chamar o serviço.

Status do pedido (Sankhya → FC)

Faturamento/expedição alteram TGFCAB / TGFVOL → NotaFiscalListener / TrackingListener enfileiram status/tracking. Catch-up: OrderStatusSyncJob (status-sync , 3 min) lê AD_FCMAP (STATUS_SKW vs STATUS_FC) e sincroniza divergências. Mapa Sankhya→FC (OrderStatusSyncJob:38-42 , códigos em FastchannelConstants):

STATUS_SKW	Significado	Código FC
L	Liberado	201 (STATUS_APPROVED)
F	Faturado	300 (STATUS_INVOICE_CREATED)
E	Entregue	301 (STATUS_DELIVERED)
C / X	Cancelado	400 (STATUS_DENIED)

Diagrama textual do fluxo



Tabelas AD_FC* do add-on (README.md:42-48): AD_FCCONFIG (configuração, lida por FastchannelConfig), AD_FCQUEUE (fila/outbox), AD_FCPEDIDO (pedidos importados), AD_FCLEGS (logs), AD_FCDEPARA (mapeamento SKU↔CODPROD). Além destas, o código referencia AD_FCMAP (mapa de pedido: STATUS_SKW / STATUS_FC / NUNOTA) e AD_FCDUPPURGE (tabela opcional de purge de duplicatas AD_NUMFAST).

Build, Deploy e Publicação

Esta seção documenta o pipeline completo de build do addon Sankhya→FastChannel, desde `compileJava` até o artefato `addon-fastchannel.exts` publicado no Sankhya Place. O projeto usa **Gradle** com o plugin oficial `br.com.sankhya.addonstudio:2.0.0`, sobre **JDK 8** (`C:\Program Files\Eclipse Adoptium\jdk-8.0.462.8-hotspot`, ver `run_build.bat:2`).

AVISO CENTRAL: o `build.gradle` da raiz (1074 linhas) é o coração do processo. O plugin AddonStudio gera artefatos *quebrados/incompletos* por padrão neste cenário (JAPE, menu, license, WAR duplicado, página inicial). Praticamente todo o `build.gradle` são *hooks de pós-processamento* que corrigem a saída do plugin. **Não simplifique nem remova esses hooks sem entender exatamente o que cada um conserta** — cada bloco tem um comentário `[CRIT-*/ [FIX ...]` referenciando um incidente real de produção.

1. Estrutura de módulos e wiring de versão

`settings.gradle` (linhas 7-9) define três projetos Gradle:

```
rootProject.name = 'addon-fastchannel' // OBRIGATÓRIO kebab-case lowercase (define o prefixo dos descritores META-INF)
include 'model' // camada de negócio (Java: br.com.bellube.fastchannel.*)
include 'vc' // camada view/control (webapp, gera o WAR)
```

- `model/build.gradle` — praticamente vazio (só bloco `dependencies {}`); as dependências de teste (`junit`, `mockito-core`, `h2`, `bsh`) são injetadas de fora, pela raiz em `build.gradle:455-461` (`project(':model') { dependencies { ... } }`).
- `vc/build.gradle` — apenas `implementation project(":model")` (linha 3). O `vc` é quem produz o WAR; depende do `model`.
- `gradle.properties` — só tuning de JVM: `org.gradle.jvmargs=-Xmx2g` + timeouts de socket longos (`defaultReadTimeout=600000`), necessários porque o `publishAddon` faz upload HTTP grande para a Área Dev.

Coordenadas e versão (`build.gradle:26-30`):

- `group = 'br.com.bellube.fastchannel'`
- `version = (project.findProperty('ADDON_VERSION') ?: System.getenv('ADDON_VERSION') ?: "1.2.91")` ← a versão default é **1.2.91**, mas pode ser sobreposta por `-PADDON_VERSION=x.y.z` ou variável de ambiente `ADDON_VERSION`.
- `description = "Integração Fastchannel Data Lake - Monitoramento e Ingestão de Dados"`

Configuração do addon (`build.gradle:51-63`):

- `appKey="b262dd0f-3ca5-4219-8053-7a492f792590"` — APPKEY original que funciona em produção (registrada no Portal do Desenvolvedor; alterá-la quebra a assinatura).
- `parceiroNome = "BEL DISTRIBUIDOR DE LUBRIFICANTES LTDA"`
- `autoDDL=false` — as tabelas vêm de scripts manuais em `dbscripts/V*.xml`, **não** são geradas a partir do datadictionary.
- `plataformaMinima = "4.28"` (`build.gradle:48`, dentro de `snkmodule`).

Config por máquina (`build.gradle:34-49`): `serverFolder` (pasta do WildFly, usado só por `deployAddon`) vem de `local.properties` (`snk.serverFolder=<path>`) ou da env `SNK_SERVER_FOLDER`. Se ausente, só emite `WARNING` (build normal não precisa dele; só o deploy local precisa).

2. Tasks Gradle principais

As tasks `compileJava`, `war`, `configureAddon`, `gerarAddon`, `publishAddon`, `deployAddon`, `convertMetadata`, `createExtensionFile` são **fornecidas pelo plugin** `br.com.sankhya.addonstudio:2.0.0`. O `build.gradle` da raiz **estende** cada uma com `doLast` / `dependsOn` / `finalizedBy`. Resumo do que cada uma faz *neste projeto* (comportamento base + hooks):

Task	Origem	O que faz aqui
<code>compileJava</code>	Gradle/Java	Compila <code>model/src/main/java/br/com/bellube/fastchannel/**</code> e <code>vc</code> . JDK 8.
<code>war (:vc:war)</code>	plugin/war	Empacota o WAR <code>addon-fastchannel-web.war</code> . A raiz adiciona <code>dependsOn</code> para <code>patchExtension</code> , <code>prepareServiceProviders</code> , <code>processDashboards</code> e copia <code>dbscripts/V*.xml</code> para dentro do WAR (<code>build.gradle:438-453</code>). <code>duplicatesStrategy=EXCLUDE</code> .
<code>convertMetadata</code>	plugin	Gera <code>build/dist/datadictionary/metadata.xml</code> a partir do <code>datadictionary</code> . A raiz faz <code>doLast</code> (<code>build.gradle:254-405</code>) para enriquecer o metadata com tabelas/campos/instâncias FastChannel garantidos (<code>AD_FCONFIG</code> , <code>AD_FCQUEUE</code> , <code>AD_FCDEPARA</code> , <code>AD_FCPEDIDO</code> , <code>AD_FCLOG</code>).
<code>createExtensionFile</code>	plugin	Gera <code>build/dist/extension.xml</code> . A raiz <code>patchExtension</code> (<code>build.gradle:66-128</code>) o corrige (ver abaixo).
<code>configureAddon</code>	plugin	Monta a estrutura do addon em <code>build/dist/</code> (ejb, lib, web, META-INF, datadictionary). A raiz: (a) apaga WARs extras em <code>dist/web</code> deixando só <code>addon-fastchannel-web.war</code> (<code>build.gradle:131-141</code>); (b) roda <code>stripJarsFromWar</code> (CRIT-JAPE); (c) roda <code>injectMitraMetadataIntoWar</code> ; (d) <code>dependsOn :vc:war</code> .
<code>gerarAddon</code>	plugin	Gera o pacote final do addon (<code>build/libs/addon-fastchannel.exts</code>). Recebe os mesmos hooks CRIT-JAPE + MITRA da <code>configureAddon</code> / <code>publishAddon</code> . É a task para gerar o artefato local sem publicar.
<code>publishAddon</code>	plugin	Faz upload do addon para a Área Dev Sankhya (<code>areadev.sankhya.com.br</code>). <code>dependsOn</code> <code>patchExtension</code> , <code>syncDbScripts</code> , <code>injectExtensionIntoWar</code> (<code>build.gradle:860-866</code>); <code>doNotTrackState(...)</code> para não falhar por lock de arquivo no Windows após o upload; recebe os hooks CRIT-JAPE + MITRA; e <code>finalizedBy 'releaseAreaDevVersion'</code> (<code>build.gradle:963-965</code>).
<code>releaseAreaDevVersion</code>	custom (<code>build.gradle:868-961</code>)	Após o publish, promove a versão de <code>PUBLISHED</code> para <code>RELEASED</code> na Área Dev via REST (ver §6).
<code>deployAddon</code>	plugin (redefinida)	Redefinida na raiz (<code>build.gradle:782-858</code> , <code>actions.clear()</code>): copia <code>build/dist/</code> para <code>standalone/deployments/addon-fastchannel.ear</code> do WildFly local e cria o marcador <code>.dodeploy</code> , aguardando <code>.deployed</code> / <code>.failed</code> (timeout 5 min). É o deploy local/homolog (não o de produção).
<code>injectExtensionIntoWar</code>	custom (<code>build.gradle:546-780</code>)	Re-empacota o WAR final aplicando ~15 correções (service-providers, extension-listeners, web.xml, jboss-web.xml context-root <code>/addon-fastchannel</code> , metadata em múltiplos paths, remoção de <code>license.properties</code> residual, etc.).
<code>patchExtension</code>	custom (<code>build.gradle:66-128</code>)	Garante <code><display-name>Fastchannel Integration</display-name></code> e <code><vendedor></code> no <code>extension.xml</code> ; opcionalmente injeta <code>ADDON_LICENSE_ID</code> em <code><vendedor><id></code> e <code><license-by-resource><default-module></code> .
<code>prepareServiceProviders / syncDbScripts / processDashboards</code>	custom	Copiam <code>service-providers.xml(.manual)</code> , <code>dbscripts/V*.xml</code> e configs de dashboards para os locais que o loader espera.
<code>prepareDeployDir</code>	custom (<code>build.gradle:408-433</code>)	Limpa o <code>.ear</code> e marcadores antigos no <code>deployments</code> antes de um <code>deployAddon</code> .

Encadeamento efetivo do publish: `publishAddon` → (deps) `patchExtension` + `syncDbScripts` + `injectExtensionIntoWar` → (via `injectExtensionIntoWar`) `:vc:buildWar` / `:vc:war` → `patchExtension`; depois do publish, `doLast` roda `injectMitraMetadataIntoWar` no WAR final e `finalizedBy releaseAreaDevVersion`.

3. Fix CRIT-JAPE — `stripJarsFromWar` remove `libs-master.jar` do WAR

Documentado no comentário `build.gradle:143-166` e implementado em `build.gradle:169-218`.

Sintoma: `ClassCastException ... JapeSessionLifecycleImpl cannot be cast to ... JapeSessionLifecycle` em `Jape.parametersLoaded:342`, quebrando a inicialização do provider JAPE (`mge-core`) do addon.

Causa raiz (`build.gradle:143-153`): o plugin `AddonStudio` empacota `libs-master.jar` em `WEB-INF/lib/` do WAR. Esse jar contém a **interface** `br.com.sankhya.jape.core.JapeSessionLifecycle`. Como o WAR tem classloader **isolado**, ele carrega sua própria cópia da interface. A implementação `JapeSessionLifecycleImpl` (fornecida pelo `erpcore.ear` do WildFly, no classloader **pai**) implementa a interface do pai — e **não pode ser castada** para a cópia duplicada dentro do WAR. `libs-master.jar` só é necessário em *compile-time*; em runtime o WildFly já o fornece via `erpcore.ear`.

Fix: a lista `BLOCKED_RUNTIME_JARS = ['libs-master.jar']` (`build.gradle:164-166`) e a closure `stripJarsFromWar` (`build.gradle:194-218`) descompactam o WAR, deletam `WEB-INF/lib/libs-master.jar` e recompactam. O `stripAction` (`build.gradle:169-192`) roda em `doLast` de `:vc:war`, `:vc:buildWar`, `configureAddon`, `gerarAddon` e `publishAddon`, varrendo os diretórios `vc/buildGradle/libs`, `build/dist/web`, `build/tmp/warWithExtension/WEB-INF/lib` e `build/dist/lib`.

ATENÇÃO — divergência com a memória histórica: o CLAUDE.md/memória menciona remover **dois** jars (`libs-master.jar` + `mge-modelcore-master.jar`). No código **atual** (`build.gradle:154-160`), o `mge-modelcore-master.jar` **NÃO é mais removido** — o comentário explica que ele contém as VOs de domínio (`CabecalhoNotaVO`, `ItemNotaVO`, `ParceiroVO`), o `DynamicEntityLoader` do provider DWF e os wrappers `sun.proxy` do

broker EJB usados pelo `ServiceInvokerStrategy`. Removê-lo quebra homolog com `Could not initialize class com.sun.proxy.$Proxy1613` e Nenhum provedor encontrado para CACSP.incluirNota. **Só `libs-master.jar` é bloqueado hoje.**

4. Formato MITRA do `metadata.xml` + `license.properties`

4.1 Formato MITRA (visibilidade do menu)

Documentado no comentário `build.gradle:967-980`.

Problema: o plugin `SankhyaStudio:2.0.0` gera `metadata.xml` com root `<MetadadosExtensionXML>`, que o **core do Sankhya não processa para o menu** de usuários não-SUP → só o SUP enxerga o addon, mesmo com TDDPER configurado.

Solução: substituir o `metadata.xml` gerado pelo **template MITRA** `metadata-mitra-template.xml` (raiz do projeto). Esse template usa:

- root `<metadata>` (não `<MetadadosExtensionXML>`);
- `<controls>` com FQCN prefixado `br.com.bellube.fastchannel.*` (`framebuilder`, `module`, `dashboard`, `pedidos`, `estoque`, `precos`, `config`, `logs`);
- `<showMenu><![CDATA[true]]></showMenu>` e `<contexto>addon-fastchannel</contexto>` em cada controle de menu;
- `<controlRelations>` ligando o `module` a cada tela;
- URLs no padrão `/addon-fastchannel/html5/fastchannel/<tela>.html?mgeSession=${mge.session.id}&resourceID=${resourceID}`;
- O template também declara os **extension fields em `TGFCAB`** (tabela nativa): `AD_DESCONTO_FAST` (F), `AD_NUMFAST` (S), `AD_FASTCHANNEL_ID` (S), `AD_PARCELAS_FAST` (I) — necessários para o VO de `CabecalhoNota` expor esses atributos (senão a importação FC falha com *"Propriedade AD... não existe para o ValueObject CabecalhoNota"*). Ver comentários `metadata-mitra-template.xml:36-62`.

Validado 17/04/2026 com o usuário comum `rodrigo_f` (CODUSU=341) vendo o addon no menu.

Implementação (dois pontos, ambos idempotentes):

1. Task `convertMetadataToMitraFormat` (`build.gradle:981-998`), `dependsOn convertMetadata`: sobrescreve `build/dist/datadictionary/metadata.xml` com os bytes do template (`target.bytes = template.bytes`).
2. Closure `injectMitraMetadataIntoWar` (`build.gradle:1008-1056`): descompacta o WAR final e injeta (a) `WEB-INF/dictionary/metadata.xml` no formato MITRA, (b) `META-INF/extension.xml` canônico (sem prefixo, exigido pelo `AddonModuleServletContextListener`), (c) remove `license.properties` residual.
3. O hook `build.gradle:1059-1070` liga `convertMetadataToMitraFormat` (via `dependsOn`) e `injectMitraMetadataIntoWar` (via `doLast`) em `configureAddon`, `gerarAddon` e `publishAddon`.

4.2 `license.properties` — o que NÃO fazer

Documentado nos comentários `build.gradle:747-759` e `build.gradle:1039-1048` (blocos de geração **REMOVIDOS em 20/04/2026**).

Regra de ouro: o addon NÃO deve gerar `license.properties`. Uma versão antiga do build gerava `META-INF/license.properties` com `webHash = MD5(files)`. Mas o validador de runtime `br.com.sankhya.addonmodule.1.h` espera `webHash = MD5(signed.jar + codParc + "SKWLIC3NCAPAC0TEAD3DON")` — cálculo totalmente diferente → nunca batia → erro **"Assinatura do addon mudou / adquira pela Loja"**, travando o addon em produção por 4 dias.

Comportamento atual: o build **remove ativamente** qualquer `META-INF/license.properties` residual (em `injectExtensionIntoWar` linha 755-759 e em `injectMitraMetadataIntoWar` linha 1044-1048). Quem injeta o `license.properties` canônico e assina o jar é o **Sankhya Place**, durante o `publish`/instalação — não o Gradle. (Homolog funciona **sem** `license.properties` porque o `1.h` pula a validação nesse caso.)

5. Como buildar/publicar (Windows)

Pré-requisito: `JAVA_HOME` → JDK 8. O `run_build.bat` já fixa isso.

Gerar o artefato local (sem publicar):

```
cmd /c '.\gradlew.bat clean gerarAddon'
```

Produz `build/libs/addon-fastchannel.exts` (ver §7).

Gerar + publicar na Área Dev (fluxo padrão de release):

```
cmd /c '.\gradlew.bat gerarAddon publishAddon'
```

Nota sobre credenciais: o `publishAddon` do plugin exige as credenciais do Portal/Área Dev. O `run_build.bat` (linha 4) mostra o formato real usado em produção:

```
bat call gradlew.bat clean publishAddon -Pemail=suporteti@bellube.com.br -Ppassword=102030 -Ppublish=true
-Ppublish=true também habilita o releaseAreaDevVersion (§6). Sem publish=true, a promoção automática é ignorada (build.gradle:872-878).
```

Deploy local/homolog no WildFly (não é o de produção): requer `local.properties` com `snk.serverFolder=X:/Wildfly_Clean/wildfly_producao` (ou `env SNK_SERVER_FOLDER`), depois:

```
cmd /c '.\gradlew.bat deployAddon'
```

Isso copia `build/dist/` para `standalone/deployments/addon-fastchannel.ear/` e cria o `.dodeploy` (`build.gradle:838-857`).

6. Fluxo de release: Área Dev → Sankhya Place → produção

1. **Build local** gera `addon-fastchannel.exts` com todos os hooks (CRIT-JAPE + MITRA + injeções).
2. **publishAddon** faz upload do addon para `https://areadev.sankhya.com.br` (Área Dev), criando uma nova *solution-version* no estado `PUBLISHED`. A solução deste addon é `AREADEV_SOLUTION_ID = 169614` (default em `build.gradle:889-891`).
3. **releaseAreaDevVersion** (`build.gradle:868-961`, disparado por `finalizedBy` do publish) promove essa versão para `RELEASED` automaticamente via REST:
 - `GET {AREADEV_BASE_URL}/api/solution-versions/solution/169614` — lista versões, acha a entry cujo `version == project.version` (ex.: `1.2.91`) e pega o `id`.
 - `POST {AREADEV_BASE_URL}/api/solution-versions/deploy` com body `{solution_id, version_id, deploy_in_test_environment:false}`.
 - Requer `AREADEV_BEARER_TOKEN` (via `-P` ou env). Sem token: pula silenciosamente, exceto se `AREADEV_AUTO_RELEASE_STRICT=true` (ai lança `GradleException`). Só roda se `publish=true` / `PUBLISH=true`.
4. **Sankhya Place** (marketplace de addons do cliente): após a versão estar `RELEASED` na Área Dev, o Place a sincroniza (leva ~1-2 min). É o Place que **assina o jar** e injeta o `license.properties` canônico (por isso o build não deve gerá-lo — §4.2).
5. **Instalação em produção:** o cliente (base BEL, `codParc=15680`) abre o Sankhya Place e instala/atualiza o `addon-fastchannel`. A validação de assinatura `1.h` verifica `MD5(signed.jar + 15680 + salt)` contra o `webHash` que o Place gravou.

Armadilha conhecida (CHANGELOG.md:585-591): erros de "SolutionBinary"/reinstalação no Place às vezes são **internos do Place**, não do addon — mitiga-se aguardando 1-2 min após publicar nova versão na Área Dev para a sincronização do Place completar.

7. Artefato final: `build/libs/addon-fastchannel.exts`

O artefato publicável é `build/libs/addon-fastchannel.exts` (nome derivado de `rootProject.name = addon-fastchannel`). Confirmado no disco: ~4,5 MB, gerado em 15/06 pelo último build da v1.2.91.

Estrutura montada em `build/dist/` (que compõe o `.exts`):

- `extension.xml` — descriptor raiz. O `build/dist/extension.xml` gerado confirma `<version>1.2.91</version>`, `<id>addon-fastchannel</id>`, `<platform-min-version>4.28</platform-min-version>`, `<appkey>b262dd0f-3ca5-4219-8053-7a492f792590</appkey>`, `<buildApp>SankhyaStudio-2.0.0</buildApp>` e `build-timestamp` do publish.
- `datadictionary/metadata.xml` — no formato MITRA.
- `dbscripts/` — scripts `V*.xml` de criação/migração de tabelas.
- `ejb/`, `lib/` — jars de negócio (com `libs-master.jar` já removido pelo CRIT-JAPE).
- `web/addon-fastchannel-web.war` — WAR único (WARs extras do Studio deletados), context-root `/addon-fastchannel`.
- `META-INF/` — `application.xml`, `jboss-all.xml`, `jboss-app.xml`, `jboss-deployment-structure.xml`, `deployment.properties`, `module.xml` (copiados de `addon-fastchannel.ear/META-INF` — `build.gradle:766-778, 800-821`).

Quando instalado no WildFly (via Place ou `deployAddon`), o pacote é deployado como `addon-fastchannel.ear` em `standalone/deployments/`.

Arquivos-chave desta seção (caminhos absolutos):

- `X:\IntegracaoFastchannel\APPFASCHANNEL\worktrees\merge-unify\build.gradle` (1074 linhas — todos os hooks)
- `X:\IntegracaoFastchannel\APPFASCHANNEL\worktrees\merge-unify\settings.gradle`
- `X:\IntegracaoFastchannel\APPFASCHANNEL\worktrees\merge-unify\gradle.properties`
- `X:\IntegracaoFastchannel\APPFASCHANNEL\worktrees\merge-unify\vc\build.gradle`
- `X:\IntegracaoFastchannel\APPFASCHANNEL\worktrees\merge-unify\model\build.gradle`
- `X:\IntegracaoFastchannel\APPFASCHANNEL\worktrees\merge-unify\metadata-mitra-template.xml`
- `X:\IntegracaoFastchannel\APPFASCHANNEL\worktrees\merge-unify\run_build.bat`
- `X:\IntegracaoFastchannel\APPFASCHANNEL\worktrees\merge-unify\build\libs\addon-fastchannel.exts` (artefato)
- `X:\IntegracaoFastchannel\APPFASCHANNEL\worktrees\merge-unify\build\dist\extension.xml` (descriptor da v1.2.91)

Integração API FastChannel (referência)

Referência técnica da camada de comunicação com a API FastChannel Commerce. Cobre autenticação, URLs, os clientes especializados (preço, estoque, pedidos, status) e as três estratégias de criação de pedido no Sankhya. Todos os nomes, linhas e gotchas abaixo foram lidos do código em `model/src/main/java/br/com/bellube/fastchannel/`.

Arquivos desta camada

Arquivo	Papel
<code>http/FastchannelHttpClient.java</code>	Cliente HTTP base (rate limit, retry, SSL, subscription-key, renovação de token em 401)
<code>http/FastchannelPriceClient.java</code>	Price Management: PUT preço, GET/POST/DELETE batches escalonados
<code>http/FastchannelStockClient.java</code>	Stock Management: PUT/GET estoque
<code>http/FastchannelOrdersClient.java</code>	Order Management: listar/obter pedidos, status, NF, tracking, sync
<code>auth/FastchannelTokenManager.java</code>	Singleton OAuth2 client_credentials (cache + renovação proativa)
<code>config/FastchannelConstants.java</code>	URLs base, endpoints, status codes, nomes de tabelas
<code>config/FastchannelConfig.java</code>	Carrega credenciais/parâmetros de <code>AD_FCCONFIG</code>
<code>service/strategy/OrderCreationOrchestrator.java</code>	Orquestra as 3 estratégias com fallback + idempotência
<code>service/strategy/ServiceInvokerStrategy.java</code>	Estratégia 1 (XML nativo via <code>CACSP.incluirNota</code>)
<code>service/strategy/InternalApiStrategy.java</code>	Estratégia 2 (JAPE direto)
<code>service/strategy/HttpServiceStrategy.java</code>	Estratégia 3 (HTTP com login/JSESSIONID)
<code>service/auth/SankhyaAuthManager.java</code>	Login/logout Sankhya para a estratégia HTTP

1. Autenticação OAuth2 (Azure AD)

- O token é obtido via `FastchannelTokenManager` (`auth/FastchannelTokenManager.java`), Singleton thread-safe (`getInstance()`, linha 63) com cache em memória do `access_token` + `expiresAt`.
- Endpoint** (`FastchannelConstants.AUTH_URL`, linha 18): `https://login.microsoftonline.com/fastchannel.com/oauth2/v2.0/token`. Pode ser sobrescrito por `AD_FCCONFIG.AUTH_URL` (`FastchannelConfig.getAuthUrl()`).
 - Grant type:** `client_credentials`. O corpo é `application/x-www-form-urlencoded`, montado em `renewToken()` (linha 141): `grant_type=client_credentials&client_id=<CLIENT_ID>&client_secret=<CLIENT_SECRET>&scope=<SCOPE>` — todos URL-encoded via `encode()`.
 - Credenciais:** `CLIENT_ID`, `CLIENT_SECRET`, `SCOPE` lidos de `AD_FCCONFIG` (`FastchannelConfig.applyFromResultSet`, linhas 136-138). Se qualquer um estiver vazio, `renewToken()` lança exceção antes da chamada HTTP (linhas 130-138).
 - Cache/renovação:** `getValidToken()` (linha 77) reusa o token enquanto `now < (expiresAt - buffer)`. O buffer é `TOKEN_REFRESH_BUFFER_SECONDS = 300` (5 min, `FastchannelConstants` linha 69). `expiresAt = now + (expires_in * 1000)` (linha 185).
 - Renovação em 401:** quando qualquer chamada recebe HTTP 401, `FastchannelHttpClient.executeWithRetry` chama `tokenManager.forceRenew()` e repete (linhas 194-199 de `FastchannelHttpClient`).
 - Parsing robusto do token:** `parseTokenResponse` (linha 229) normaliza chaves (remove BOM/zero-width `/`) e tem fallback via regex para `access_token` / `expires_in` (linhas 254-259), tolerando respostas "sujas" do gateway.
 - SSL:** `configureSslIfNeeded` (linha 334) — atenção: no `TokenManager` o default é **inseguro** (trust-all) quando a propriedade não está setada (`insecure = configured == null || ... || parseBoolean`, linha 343). No `FastchannelHttpClient` o comportamento inverso: só fica inseguro se explicitamente ligado (linha 429). Controlado por `-Dfastchannel.ssl.insecure` ou env `FASTCHANNEL_SSL_INSECURE`.

2. URLs base e subscription keys

URLs em `FastchannelConstants` (linhas 21-27):

API	Constante	Valor
Order Management	<code>ORDER_API_BASE</code>	<code>https://api.commerce.fastchannel.com/order-management/v1</code>
Stock Management	<code>STOCK_API_BASE</code>	<code>https://api.commerce.fastchannel.com/stock-management/v1</code>
Price Management	<code>PRICE_API_BASE</code>	<code>https://api.commerce.fastchannel.com/price-management/v1</code>

Order pode ser sobrescrito por `AD_FCCONFIG.BASE_URL` apenas se contiver `/order-management/` (`FastchannelHttpClient.buildOrderUrl`, linhas 104-110). Stock e Price sempre usam as constantes.

Duas subscription keys por canal

`AD_FCCONFIG` guarda `SUBSCRIPTION_KEY_DISTRIBUTION` e `SUBSCRIPTION_KEY_CONSUMPTION` (mais o legado `SUBSCRIPTION_KEY` como fallback). Em `FastchannelConfig` (linhas 230-235) cada key vazia cai para `subscriptionKey`. Getters: `getSubscriptionKeyDistribution()` (312), `getSubscriptionKeyConsumption()` (320).

Mapeamento por rota no `FastchannelHttpClient`:

- **Distribution** — Orders (`getOrders` / `postOrders` / `putOrders` , linhas 85/93/101) e **Stock** (`getStock` / `putStock` , linhas 118/127).
- **Consumption** — Price por padrão (`getPrice` / `putPrice` / `postPrice` , linhas 134/141/148).

O `FastchannelPriceClient` tem enum `Channel {DISTRIBUTION, CONSUMPTION}` (linha 30) e resolve a key em `getSubscriptionKeyForChannel()` (linha 867); default é `CONSUMPTION` (construtores linhas 64-80).

Como a key vai na request (gotcha de header duplicado)

Em `doHttpCall` (linhas 346-368) a key é enviada de **três** formas simultâneas para satisfazer o gateway APIM da FastChannel/Vertis:

1. Query param `?subscription-key=<key>` (linha 350).
2. Header `Subscription-Key` (linha 362).
3. Header `Ocp-Apim-Subscription-Key` (linha 363).

Gotcha documentado no código (2026-04-20, linhas 364-368): foi **removido** o `setRequestProperty("subscription-key", ...)` porque nomes de header HTTP são case-insensitive → `Subscription-Key` + `subscription-key` viravam o **mesmo** header com valor duplicado `"X,X"`, e o gateway Vertis rejeitava com `403 "chave de assinatura '...',...' informada nao e valida"`.

Demais headers fixos: `Authorization: Bearer <token>`, `Accept: application/json`, `Content-Type: application/json` (quando há corpo). Conexão sempre com `Proxy.NO_PROXY` (linha 352).

Rate limit e retry (base para todas as chamadas)

- **Rate limit:** sliding window de 60 s (`WINDOW_SIZE_MS` , linha 56), limite `AD_FCCONFIG.MAX_REQUESTS_MIN` (default `DEFAULT_RATE_LIMIT_PER_MINUTE = 30`). Implementado em `waitForRateLimit()` (linha 319).
- **Retry:** `MAX_RETRIES = 3` (`DEFAULT_MAX_RETRIES`), backoff exponencial (500 ms base, mult. 2.0, jitter ±20%) em `calculateBackoff` (linha 303). Retenta apenas **transientes**: 429, 5xx, `SocketTimeout` / `Connect` / `NoRouteToHost` / `UnknownHost` , `IOException` com "Connection reset/refused"/"timed out"/"Broken pipe" (`isRetryable` , linha 273). 4xx (exceto 429) retornam sem retry.
- **Classificação de erro:** `classifyHttpError` (linha 251) → 401/403= `FastchannelAuthException` , 429= `FastchannelRateLimitException` , 5xx= `FastchannelTransientException` , demais 4xx= `FastchannelFatalException` . Header `Retry-After` (segundos) é lido (linha 390).

3. PREÇO — `PUT /prices/{sku}`

Endpoint `ENDPOINT_PRICE = "/prices/%s"` (`FastchannelConstants` linha 42). Implementado em `FastchannelPriceClient.updatePrice` (linhas 89 / 101 / 142).

Payload (`LinkedHashMap` , ordem preservada, linhas 114-122):

```
{ "ResellerId": "<id>", "PriceTableId": <int>, "SalePrice": <int>, "ListPrice": <int> }
```

- **SalePrice e ListPrice são enviados como INTEIRO** — `price.intValue()` / `effectiveListPrice.intValue()` (linhas 121-122). São valores em centavos.
- **Regra `ListPrice >= SalePrice`** (linhas 106-112 e 163-168): a FastChannel **rejeita** `SalePrice > ListPrice` . O cliente detecta isso e força `effectiveListPrice = price` (log: "Ajustando ListPrice = SalePrice para evitar rejeicao FC"). Se `listPrice` for `null` , usa `price` .
- Consulta: `getPrice(sku)` / `getPrice(sku, preferredPriceTableId)` (linhas 758-778); 404 → `null` . Parser em `parsePriceFromResponse` (linha 795) — lê `Payload` (array ou objeto), com match opcional por `PriceTableId` .
- Descoberta de tabelas: `listPriceTables()` (linha 572) via `GET /prices?PageSize=5000` (a FC **não expõe** endpoint `/tables` ; 404). Complementa com `AD_FCCONFIG.PRICE_TABLE_IDS` . Listagem por tabela: `listPricesForTable(priceTableId)` via `GET /prices?PriceTableId=X&PageNumber=&PageSize=` paginando por `TotalPages` , lendo `ProductId` / `SalePrice` / `ListPrice` do `Payload` (linha 652).

4. ESCALONADO / batches — `GET / POST / DELETE /prices/{sku}/batches`

Endpoint `ENDPOINT_PRICE_BATCHES = "/prices/%s/batches"` (`FastchannelConstants` linha 43). O `%s` é o **SKU** (não o reseller — apesar de `updatePricesBatch` na linha 219 usar `resellerId` , esse método é legado; o fluxo real é `updatePriceBatches(sku, priceTableId, batches)` , linha 245). DELETE: `.../batches/{batchId}` (`deletePriceBatch` , linha 484).

Reconciliação completa em `updatePriceBatches` (linha 245): lista o estado atual, compara com o desejado (`normalizeDesiredBatches` + `filterBatchesByPriceTable`), mantém uma ocorrência de cada faixa, deleta lixo/duplicata/obsoleto e faz POST só do que falta. Há um cache `NO_BATCH_SKU_TABLE_CACHE` (linha 57) para pular o GET quando o par SKU+tabela já foi confirmado sem batches (evita ~12.000 GETs num full-sync).

Gotcha (a) — XML sem Accept vs JSON com Accept, e o array vem em Payload

O `FastchannelHttpClient` sempre envia `Accept: application/json` (linha 361), então a FC (backend ServiceStack) responde JSON. **O array de batches vem no campo Payload , NÃO em ProductPriceBatch** (que é o nome do ELEMENTO no XML que a FC retornaria se não houvesse `Accept: application/json`).

`listPriceBatches` (linha 414) foi corrigido em **2026-05-25 (v1.2.85)** (comentário linhas 430-437): lê `root.getAsJsonArray("Payload")` (linha 447), com fallback legado para `ProductPriceBatch` (linha 449). O parser antigo lia só `ProductPriceBatch` → **sempre retornava lista vazia** → o dedup/cleanup ficava cego → re-POST a cada sync. Também remove BOM inicial (`stripBom`, linha 477) e loga WARNING se não achar array reconhecível (não mascara como vazio — linhas 466-472).

Gotcha (b) — quantidade DECIMAL grava Min/Max = 0

A FC grava `MinimumBatchSize` / `MaximumBatchSize` como 0 se receber notação decimal (`"1.0"`, `"3.0"`) em vez de inteiro (`"1"`, `"3"`). Fix **2026-05-26 (v1.2.86)** em `normalizeDesiredBatches` (linhas 504-514): `setScale(0, RoundingMode.HALF_UP)` em Min e Max — chokepoint único de todo POST. Causa raiz: `TGFDPO.QTDE` é float no SQL → `getBigDecimal()` devolvia `1.0` → Gson serializava `"1.0"`. Confirmado contra a API real: POST `2.0` → FC grava Min=0; POST `2` → grava Min=2.

Gotcha (c) — FC NÃO deduplica POST

Dois POST idênticos = dois batches (a FC não deduplica). Incidente 2026-05-25: SKU `31251453` teve a mesma faixa 1-3 enviada 6× porque, com o parser de `Payload` quebrado (gotcha a), o cliente nunca via os batches existentes. Mitigação em `updatePriceBatches` (linhas 273-335):

- `isGarbageBatch` (linha 339): `Min≤0 OU Max≤0 OU BatchDisabled=true OU preço≤0` → DELETE.
- `isValidBatchPayload` (linha 353): `Min≥1, Max≥Min, preço>0, PriceTableId` definido → só posta se válido.
- `containsEquivalentBatch` / `isEquivalentBatch` (linhas 520/532): compara `PriceTableId`, Min, Max, `UnitaryPriceForBatch`, `BatchDisabled` — para não recriar o que já existe (lista `kept`) e para deletar duplicatas.
- `cleanupGarbageBatches(sku)` (linha 372): ferramenta admin que deleta batches lixo em todas as tabelas.

Campos JSON do batch (via `PriceBatchItemDTO`): `PriceTableId`, `MinimumBatchSize`, `MaximumBatchSize`, `UnitaryPriceForBatch`, `BatchDisabled`, `BatchId`.

5. PEDIDO — 3 estratégias + orquestrador

Interface `OrderCreationStrategy` (`service/strategy/OrderCreationStrategy.java`): `getStrategyName()`, `isAvailable()`, `createOrder(order, codParc, codTipVenda, codVend, codNat, codCenCus)` → retorna `NUNOTA`.

O `OrderCreationOrchestrator` (linha 33) tenta em **ordem fixa** com fallback automático:

1. `ServiceInvokerStrategy` (XML nativo — preferencial)
2. `InternalApiStrategy` (JAPE direto — fallback 1)
3. `HttpServiceStrategy` (HTTP com login — fallback 2)

A ordem foi trocada em **2026-05-14 (v1.2.77)** (comentário linhas 19-25): `ServiceInvoker` virou primária porque a abordagem JAPE do `InternalApi` conflitava com o recálculo nativo `Sankhya` (`MGECOM/STP_CONFIRMANOTA2`), exigindo 4 camadas de "repair" para o cupom de desconto.

Idempotência (CRIT-1, linhas 61-70): antes de qualquer estratégia, `findExistingNuNotaByAdNumFast` (linha 124) faz `SELECT TOP 1 NUNOTA FROM TGFCAB WHERE AD_NUMFAST = ?` (o `order.getOrderId()`). Se já existe, reusa o `NUNOTA` — resolveu 1981 notas duplicadas no homolog (33%) que ocorriam quando uma estratégia comitava mas a resposta perdia o `NUNOTA`, fazendo o orquestrador cair para a próxima e criar OUTRA nota. É fail-open (erro de leitura não bloqueia).

Cada estratégia é tentada só se `isAvailable()`; se lançar, o erro é acumulado e passa-se à próxima. Se todas falham, lança com o relatório de `buildErrorReport` (linha 150).

ServiceInvokerStrategy (`SERVICE_NAME = "CACSP.incluirNota"`)

Monta XML via `OrderXmlBuilder.buildIncluirNotaXml` e invoca `SankhyaNativeServiceCaller.invoke` (linha 79). Extrai `NUNOTA` por regex `<NUNOTA>(\d+)</NUNOTA>` (linha 129). Fix **2026-04-28** (linhas 64-101): abre `JapeSession.open()` + `ensureRequiredSessionProperties()` antes de tocar o modelcore, senão `MGEFrontFacadeBean.ejbCreate` quebra com "Gerenciador de sessao nao foi iniciado" (4638+2415 erros em 10h de PROD). Fail-open: se `open()` falhar, segue sem sessão.

InternalApiStrategy (JAPE direto)

`isAvailable()` retorna `EntityFacadeFactory.getCoreFacade() != null`, com `catch (Throwable)` (CRIT-2, linha 72) para capturar `NoClassDefFoundError` / `LinkageError` de conflito de classloader — sem isso o orquestrador não conseguia cair para a próxima estratégia. `createOrder` roda em `execWithTX` (linha 97): cria cabeçalho (`CabecalhoNota` via `JapeFactory`), itens, enriquece paridade com legado e força `VLRNOTA` correto (`forceVlrNotaCorrect`, fix 2026-04-30 — cupom de desconto). Grava `AD_NUMFAST = order.getOrderId()` (linha 257), e `VLRJURO` / `AD_PARCELAS_FAST` para encargo de parcelamento de cartão.

HttpServiceStrategy — o segredo do login /mge/ + serviço /mgecom/

Último recurso. Só disponível se `SANKHYA_SERVER_URL` e `SANKHYA_USER` estiverem em `AD_FCCONFIG` (`isAvailable`, linha 54). Usa `SankhyaAuthManager` para login/logout e obter o `JSESSIONID`.

O segredo (breakthrough HttpServiceStrategy): login e serviço rodam em **módulos WildFly diferentes**, mas a sessão é compartilhada se o `JSESSIONID` for propagado. As combinações estão em `LOGIN_SERVICE_COMBINATIONS` (linhas 30-38), e a **primeira e confirmada funcional** (homolog e prod) é:

- **Login** em `/mge/service.sbr` → `serviceName=MobileLoginSP.login` (`SankhyaAuthManager`, `LOGIN_SERVICE` linha 26, `SERVICE_PATH` linha 28). Retorna o `JSESSIONID` (extraído do body XML `<jsessionid>` / `<JSESSIONID>` / `"mgeSession"` / header `Set-Cookie`, `extractJsessionId` linha 211).
- **Serviço** `CACSP.incluirNota` em `/mgecom/service.sbr` (`SERVICE_PATH_MGECOM` linha 28 de `HttpServiceStrategy`).

- **Propagação da sessão:** em `invokeServiceAtPath` (linha 128) o JSESSIONID vai como **query param** `&mgeSession=<jsessionId>` (linha 131) **E** como cookie/header (`Cookie: JSESSIONID=...`, header `JSESSIONID`, linhas 140-141). É essa propagação do `mgeSession` que faz a sessão WildFly de `/mge/` valer em `/mgecom/`.

Detalhes: XML enviado como `text/xml; charset=ISO-8859-1` (bytes em ISO-8859-1, linha 148); erro Sankhya detectado por `status="0"` no corpo (linha 171), mensagem extraída de `<statusMessage>` (pode ser Base64) via `extractErrorMessage` (linha 204); NUNOTA por regex `<NUNOTA>(\d+)</NUNOTA>` (linha 193). Após cada tentativa, `logout` (com `&mgeSession=` também, `SankhyaAuthManager.logout` linha 151). As combinações restantes (`/mgecom` + `/mgecom`, `/mgecom` + `/mge`, `/mge` + `/mge`) são fallbacks tentados em sequência.

6. STATUS de pedido — campo `OrderStatusId` (não `status`)

`FastchannelOrdersClient.updateOrderStatus(orderId, status, message)` (linha 194) → `PUT /orders/{id}/status` (`ENDPOINT_ORDER_STATUS = "/orders/%s/status"`). Serializa `OrderStatusDTO`.

Gotcha crítico (fix 2026-05-19 v1.2.83, `OrderStatusDTO.java`): o campo Java `status` é anotado `@SerializedName("OrderStatusId")` (linha 26) e `message` como `@SerializedName("Message")` (linha 29). Sem isso, o Gson enviava `{"status":201,"message": "..."}` , mas a FC exige `{"OrderStatusId":201,"Message": "..."}` . Com o nome errado, `OrderStatusId` chegava `0` (default `int`) e a FC respondia `HTTP 400 "O código de status informado no parametro 'OrderStatusId' nao e um código valido"` — mensagem enganosa que parecia erro de máquina de estados, mas era só nome de campo errado no JSON. Confirmado contra o legado Node.js `gbi-app-integrador Pedidos.js:144-147`.

Códigos de status FastChannel (`FastchannelConstants` , linhas 47-52)

Constante	Valor	Significado
<code>STATUS_CREATED</code>	200	Criado
<code>STATUS_APPROVED</code>	201	Aprovado
<code>STATUS_INVOICE_CREATED</code>	300	NF criada / Faturado
<code>STATUS_DELIVERED</code>	301	Entregue
<code>STATUS_RETURNED</code>	303	Devolvido
<code>STATUS_DENIED</code>	400	Negado

Mapa status Sankhya — FastChannel

Em `OrderStatusSyncJob.mapSankhyaStatusToFc` (linha 35): a coluna `AD_FCPEDIDO.STATUS_SKW` (status Sankhya) é mapeada para o código FC:

- `"L"` (Liberado) → **201** (`STATUS_APPROVED`)
- `"F"` (Faturado) → **300** (`STATUS_INVOICE_CREATED`)
- `"E"` (Entregue) → **301** (`STATUS_DELIVERED`)
- `"C"` / `"X"` (Cancelado) → **400** (`STATUS_DENIED`)
- default: tenta parsear numericamente (caso `STATUS_SKW` já seja o código FC).

O job só envia (`PUT ../status`) quando `STATUS_FC` (último enviado) diverge do alvo. Helpers de conveniência no client: `approveOrder` →201, `markAsDelivered` →301, `denyOrder` →400 (linhas 310-330).

Sync/marcação: `markAsSynced(orderId, externalId)` → `PUT /orders/{id}/sync` com `{"IsSynched":true,"ExternalId": "<NUNOTA>"}` (linha 267); `markAsUnsynced` devolve ao pool (`{"IsSynched":false,"ExternalId":""}`, linha 292). A listagem (`listOrdersWithMeta`, linha 91) usa `CreatedAfter` com **safety overlap de 72h** (`SAFETY_OVERLAP_MS`, linha 89) e nunca envia `IgnoreCreationDate=true` junto (senão a API ignora todos os filtros de data). Só filtra `IsSynched=false` se `AD_FCCONFIG.SYNC_STATUS_ENABLED='S'`.

7. ESTOQUE — SKU do produto e prefixo/normalização

`FastchannelStockClient` (`http/FastchannelStockClient.java`): `PUT /stock/{sku}` (`ENDPOINT_STOCK = "/stock/%s"`, `FastchannelConstants` linha 39). Payload legacy-compatible em `buildLegacyCompatiblePayload` (linha 118):

```
{ "StorageId": <int>, "Quantity": <int>, "MinimumQuantity": 1,
  "HandlingTime": 0, "IsExternalStockEnabled": false, "ExternalStockHandlingTime": 0 }
```

`StorageId` vem de `AD_FCCONFIG.STORAGE_ID` (ou override), obrigatório (linha 66). `Quantity` é `intValue()`. GET: `getStock(sku, storageId)` acrescenta `? StorageId=` (linha 153); 404 → `null`; parser lê `Payload` (array→primeiro elemento, ou objeto) em `parseStockFromResponse` (linha 176). `zeroStock(sku)` faz `updateStock(sku, 0)`.

O SKU (regra da marca, não De-Para manual)

O `{sku}` da URL de estoque/preço é resolvido por `DeparaService.getSkuForStock(codProd)` (linha 353) — **regra da marca** `TGFMAR.AD_FAST='S'` + `AD_FASTREF`, não De-Para manual:

- `SELECT M.AD_FASTREF, P.REFFORN, P.CODPROD FROM TGFFPRO P INNER JOIN TGFMAR M ON M.CODIGO = P.CODMARCA AND M.AD_FAST = 'S' WHERE P.CODPROD = ?` (linhas 365-368).

- `computeSkuFromBrandRule`: se `AD_FASTREF='R'` e `REFFORN` não-vazio → retorna `REFFORN`; senão → `CODPROD` como string.
- Só se a marca **não** tem `AD_FAST='S'`, cai em `AD_FCDEPARA` (tipo `PRODUTO`, `INTEGRA_AUTO='S'`). **Nunca** retorna EAN/CODBARRA/REFERENCIA como SKU (linhas 336-339).
- Fallback JDBC direto (`getSkuForStockJdbc`, linha 406) quando JAPE/mge-core não inicializou; o resultado passa por `normalizeSku`.

O mesmo SKU é usado por preço, estoque, batches e listeners (`getSkuForStock` é chamado em `PriceService`, `EstoqueListener`, `PrecoListener`, `StockFullSyncJob`, `FCEstoqueService`, etc.).

Referência rápida — tabelas `AD_*` (config/persistência)

`FastchannelConstants` (linhas 56-60): `AD_FCCONFIG` (config/credenciais), `AD_FCQUEUE` (fila), `AD_FCDEPARA` (de-para), `AD_FCPEDIDO` (pedidos importados / status), `AD_FCLOG` (log). Config é carregada de `AD_FCCONFIG ORDER BY CODCONFIG DESC` com cache de 5 min (`FastchannelConfig.CACHE_TTL_MS`, linha 67).

Configuração & Modelo de Dados

Esta seção documenta todas as tabelas envolvidas na integração: as tabelas custom `AD_*` (criadas pelo DDL do addon) e as tabelas nativas Sankhya lidas/gravadas pelos serviços. Os scripts DDL canônicos ficam em `build/dist/dbscripts/mssqlserver.sql` (SQL Server) e `build/dist/dbscripts/oracle.sql`, e são empacotados no WAR em `WEB-INF/script/`. A carga é feita pelo mecanismo de "NomeObjeto/Executar" do Sankhya Add-on Studio (`SE_NAO_EXISTIR` para DDL idempotente, `SEMPRE` para UPDATE/seed).

Acesso a BD: todo o addon usa o datasource JNDI `java:/MGEDS` via `DBUtil.getConnection()` (JDBC direto) e, quando o mge-core/JAPE está pronto, via `EntityFacadeFactory.getCoreFacade().getJdbcWrapper()` + `NativeSql`. Praticamente todo método tem fallback JDBC para quando o JAPE ainda não inicializou (padrão recorrente `xxxJdbc(...)`).

1. `AD_FCCONFIG` — Configuração central (singleton por linha)

DDL: `build/dist/dbscripts/mssqlserver.sql:7-49` (+ vários `ALTER TABLE ADD` posteriores para colunas evolutivas). Lida por `FastchannelConfig` (`config/FastchannelConfig.java`), que é um **singleton com cache TTL de 5 min** (`CACHE_TTL_MS = 300_000`, linha 67). A query de carga sempre pega a linha mais recente: `SELECT * FROM AD_FCCONFIG ORDER BY CODCONFIG DESC` (`loadFromConfigTable()`, linha 119). Colunas evolutivas são lidas com guarda `DbColumnSupport.hasColumn(rs, ...)` para não quebrar em bases desatualizadas.

Coluna	Getter (FastchannelConfig)	Controla
CODCONFIG (PK, IDENTITY)	—	Chave; a linha de MAX(CODCONFIG) é a ativa
BASE_URL	<code>getBaseUrl()</code> :328	URL base da API de pedidos. Default <code>FastchannelConstants.ORDER_API_BASE</code> = <code>https://api.commerce.fastchannel.com/order-management/v1</code>
AUTH_URL	<code>getAuthUrl()</code> :333	Endpoint OAuth2. Default <code>FastchannelConstants.AUTH_URL</code> = <code>https://login.microsoftonline.com/fastchannel.com/oauth2/v2.0/token</code> (Azure AD)
CLIENT_ID	<code>getClientId()</code> :293	client_id OAuth2
CLIENT_SECRET	<code>getClientSecret()</code> :298	client_secret OAuth2
SCOPE	<code>getScope()</code> :303	scope OAuth2
SUBSCRIPTION_KEY	<code>getSubscriptionKey()</code> :308	Chave APIM legada / fallback. <code>getSubscriptionKey()</code> delega para Distribution
SUBSCRIPTION_KEY_DISTRIBUTION	<code>getSubscriptionKeyDistribution()</code> :312	Ocp-Apim-Subscription-Key para APIs de distribuição (preço/estoque/pedidos push). Se vazio, cai para <code>SUBSCRIPTION_KEY</code> (linha 230-232)
SUBSCRIPTION_KEY_CONSUMPTION	<code>getSubscriptionKeyConsumption()</code> :320	Chave para APIs de consumo (leitura de pedidos). Se vazio, cai para <code>SUBSCRIPTION_KEY</code> (linha 233-235)
RESELLER_ID	<code>getResellerId()</code> :393	Identificador do revendedor FC (usado em headers e mapeamento de estoque via <code>TIPO_STOCK_RESELLER</code>)
STORAGE_ID	<code>getStorageId()</code> :398	Identificador do storage/CD FC (<code>TIPO_STOCK_STORAGE</code>)
PRICE_TABLE_IDS	<code>getPriceTableIds()</code> :388	Lista de FC PriceTableIds (separados por <code>[;,\\s]+</code>). Ignorado no sync — o de-para <code>AD_FCDEPARA</code> é a fonte de verdade (ver <code>PriceTableResolver.resolveConfiguredOrMappedFcTableIds()</code> :187-191, comentário "existe apenas para compatibilidade")
PRICE_TABLE_TIPOS	<code>getPriceTableTipos()</code> :383	Lista de tipos <code>TGFTAB.AD_TIPO_FAST</code> (fallback legado, só usado se de-para vazio)
NUTAB	<code>getNuTab()</code> :373	Tabela de preço default (fallback quando não há de-para nem tipos)
TOP_PEDIDO	(coluna crua, não tem getter direto)	TOP para importação de pedido; seed default = 403
CODTIOPER	<code>getCodTipOper()</code> :338	TOP (Tipo de Operação) do pedido importado. Seed default = 403 (<code>UPD_AD_FCCONFIG...</code> :766; <code>FIX_CODTIOPER_403</code> :1056 migra 344→403)
TIPNEG	<code>getTipNeg()</code> :343	Tipo de negociação (CODTIPVENDA) do pedido
CODEMP	<code>getCodemp()</code> :378	Empresa default para estoque/preço quando não há de-para de empresa (ver <code>FastchannelProductFilter.appendConfiguredEmpresasFilter</code>)
CODLOCAL	<code>getCodLocal()</code> :368	Local de estoque default (fallback quando não há de-para de local)
CODNAT	<code>getCodNat()</code> :348	Natureza de operação do pedido
CODCENCUS	<code>getCodCenCus()</code> :353	Centro de custo do pedido
CODVEND_PADRAO	<code>getCodVendPadrao()</code> :358	Vendedor default. Constante de fallback <code>DEFAULT_CODVEND_PADRAO</code> = <code>281</code> ; seed do DDL grava 167 (<code>UPD_AD_FCCONFIG...</code> :768)
CODPARC_PADRAO	<code>getCodParcPadrao()</code> :363	Parceiro default para pedidos sem cliente mapeável
INTERVAL_ORDERS	<code>getIntervalOrders()</code> :283	Intervalo (min) do job de importação de pedidos. Default 5 (SMALLINT DEFAULT 5)
INTERVAL_QUEUE	<code>getIntervalQueue()</code> :288	Intervalo (min) do processamento da fila/outbox. Default 2 (SMALLINT DEFAULT 2)
ATIVO	<code>isAtivo()</code> :403	'S' liga a integração; qualquer outro valor desativa
BATCH_SIZE	<code>getBatchSize()</code> :408	Tamanho de lote de sync. Default <code>DEFAULT_BATCH_SIZE</code> = <code>50</code>
MAX_REQUESTS_MIN	<code>getMaxRequestsPerMinute()</code> :413	Rate limit por minuto. Default <code>DEFAULT_RATE_LIMIT_PER_MINUTE</code> = <code>30</code>
SYNC_STATUS_ENABLED	<code>isSyncStatusEnabled()</code> :438	Liga a sincronização de status de pedido de volta para a FC ('S')
DISABLE_DUPLICATE_CHECK	<code>isDuplicateCheckEnabled()</code> :487	'S' desliga a checagem de pedido duplicado. Pode ser sobrescrito por <code>- Dfastchannel.disableDuplicateCheck</code> / env <code>FASTCHANNEL_DISABLE_DUPLICATE_CHECK</code>
SANKHYA_SERVER_URL	<code>getSankhyaServerUrl()</code> :443	URL do próprio Sankhya para chamadas internas (login/serviços HTTP). Fallback: <code>- Dsankhya.server.url</code> → env <code>SANKHYA_SERVER_URL</code> . Nunca usar <code>127.0.0.1/localhost</code> (sessão WildFly não propaga entre <code>/mge/</code> e <code>/mgecom/</code>)
SANKHYA_USER / SANKHYA_PASSWORD	<code>getSankhyaUser()</code> :457 / <code>getSankhyaPassword()</code> :462	Credenciais de serviço para invocação interna
UI_SOURCE_DEFAULT / UI_ENABLE_SOURCE_2 / UI_ENABLE_SOURCE_3	<code>getUiSourceDefault()</code> :467 / <code>isUiEnableSource2()</code> :472 / <code>isUiEnableSource3()</code> :477	Config de "Source" na UI (<code>SourceConfig.from(...)</code> , linha 484)

Coluna	Getter (<code>FastchannelConfig</code>)	Controla
<code>EMAIL_NOTIFICACAO</code> / <code>EMAIL_HABILITADO</code> / <code>SMTP_HOST</code>	<code>getEmailNotificacao()</code> :499 / <code>isEmailHabilitado()</code> :504 / <code>getSmtpHost()</code> :509	Notificações por e-mail (<code>NotificationService</code>)
<code>LAST_ORDER_SYNC</code> / <code>LAST_PRODUCT_SYNC</code> / <code>LAST_STOCK_SYNC</code> / <code>LAST_PRICE_SYNC</code> (DATETIME2)	<code>getLast*Sync()</code> + <code>updateLast*Sync()</code> :514-532	Cursores/watermarks das sincronizações. O <code>updateLastXSync</code> persiste com <code>UPDATE AD_FCCONFIG SET <field>=?, DH_ALTERACAO=CURRENT_TIMESTAMP WHERE CODCONFIG = (SELECT MAX(CODCONFIG) ...)</code> (linha 539)
<code>TIMEOUT_MS</code> , <code>LOG_RETENTION_DAYS</code> , <code>MAX_RETRIES</code>	—	Colunas no DDL com defaults (30000/30/3); não são todas lidas por <code>FastchannelConfig</code>

Chaves de subscription hard-coded no seed (`UPD_AD_FCCONFIG_FASTCHANNEL_KEYS_AND_RULES` , `mssqlserver.sql:763-772`): Distribution = `10564b2abb5e475c84d32d2bdabbb857` , Consumption = `2729d311c289417fa6c5ff2d33835cd8` .

Comportamento de fallback quando não há linha: `setDefaultts()` (linha 252) desativa a integração (`ativo=false`) e usa constantes de `FastchannelConstants` .

2. `AD_FCDEPARA` — De-Para (mapeamentos Sankhya ↔ FC)

DDL: `mssqlserver.sql:94-103` . Colunas: `IDDEPARA` (PK IDENTITY), `TIPO_ENTIDADE` `VARCHAR(30)` , `COD_SANKHYA` `INT` , `COD_EXTERNO` `VARCHAR(100)` , `INTEGRA_AUTO` `CHAR(1)` (adicionada em :618 , default 'N' no DDL mas normalizada para 'S' pelo seed `UPD_AD_FCDEPARA_INTEGRA_AUTO_NULL` :751), `DH_CRIACAO` , `DH_ALTERACAO` . Constraints: `UK_FCDEPARA_TIPO_SKW` `UNIQUE`(`TIPO_ENTIDADE` , `COD_SANKHYA`) ; índice `IDX_FCDEPARA_EXT`(`TIPO_ENTIDADE` , `COD_EXTERNO`) .

Serviço: `service/DeparaService.java` (singleton, cache em memória `ConcurrentHashMap` com TTL 10 min, thread-safe). Tipos de entidade declarados como constantes (`DeparaService.java:39-48`):

`TIPO_PRODUTO="PRODUTO"` , `TIPO_PARCEIRO="PARCEIRO"` , `TIPO_LOCAL="LOCAL"` , `TIPO_TABELA_PRECO="TABELA_PRECO"` , `TIPO_STOCK_STORAGE="STOCK_STORAGE"` , `TIPO_STOCK_RESELLER="STOCK_RESELLER"` , `TIPO_EMPRESA="EMPRESA"` , `TIPO_TOP_PEDIDO="TOP_PEDIDO"` , `TIPO_TIPNEG="TIPNEG"` . (O DDL de seed também insere linhas `CODVEND` , `CODNAT` , `CODCENCUS` — `mssqlserver.sql:800-843` .)

`INTEGRA_AUTO='S'` habilita a integração automática daquela entidade. Todas as queries de sync filtram `COALESCE(INTEGRA_AUTO,'S')='S'` , com detecção da existência da coluna via `supportsIntegraAuto()` (cacheia consulta a `INFORMATION_SCHEMA.COLUMNS` , `DeparaService.java:1227`).

★ REGRA DO SKU (fonte primária, alinhada ao legado Node.js `fastChannel.js`)

O SKU FC **não** é o de-para manual por padrão — é derivado da **marca** do produto. Implementado em `DeparaService.getSkuForStock()` (:353) e `computeSkuFromBrandRule()` (:464), e replicado em SQL por `FastchannelProductFilter.resolveSkuExpression()` (:86) e `resolveMarcaJoin()` (:102):

- JOIN obrigatório: `INNER JOIN TGFMAR M ON M.CODIGO = P.CODMARCA AND M.AD_FAST = 'S'` — só marcas com `TGFMAR.AD_FAST='S'` participam da integração.
- Se `TGFMAR.AD_FASTREF='R'` e `TGFPRO.REFFORN` não-vazio → **SKU = REFFORN**.
- Caso contrário (`AD_FASTREF='C'` ou default) → **SKU = CODPROD** (`codProd.toPlainString()`).
- `TGFMAR.AD_FAST` e `TGFMAR.AD_FASTREF` são colunas custom criadas pelo DDL (`mssqlserver.sql:718` default 'N' / :728 default 'C' ; seed em :738). O JOIN em `resolveMarcaJoin()` filtra `AD_FASTREF IN ('C','R')` .

Ordem de resolução do SKU (`getSkuWithFallback()` :325): (1) regra da marca; (2) de-para explícito `AD_FCDEPARA` tipo `PRODUTO` . **Nunca** retorna EAN/CODBARRA/REFERENCIA como SKU (comentário explícito :336-340).

Resolução inversa SKU→CODPROD (`getCodProdBySkuOrEan()` :484, e batch `prefetchCodProdForSkus()` :690 para evitar N+1): tenta na ordem (1) `REFFORN` com marca FC **filtrando** `AD_FASTREF='R'` — este filtro é o fix v1.2.84 (`DeparaService.java:537-543`) que corrigiu bug de PROD onde SKU 12655 mapeava errado para CODPROD 14412 porque a marca MILITEC usa `AD_FASTREF='C'` ; (2) de-para `PRODUTO` ; (3) `REFERENCIA` ; (4) `REFFORN` genérico; (5) `EAN` ; (6) `CODPROD` direto.

O de-para de `TABELA_PRECO` tem tratamento especial de "família": ao gravar (`setMapping` :191), normaliza o `COD_SANKHYA` para a NUTAB vigente da mesma CODTAB (`normalizeSankhyaCodeForMapping` :1034), valida unicidade do ID FC (`validateUniqueFastCodeForPriceTable` :1046, lança "O ID Fast X já está vinculado a outra tabela...") e limpa duplicatas da família (`cleanupDuplicatePriceTableFamilyMappings` :1082).

3. `AD_FCQUEUE` — Outbox (fila de saída Sankhya → FC)

DDL: `mssqlserver.sql:59-74` . Colunas: `IDQUEUE` (PK IDENTITY), `ENTITY_TYPE` `VARCHAR(30)` NOT NULL , `OPERATION` `VARCHAR(20)` NOT NULL , `ENTITY_ID` `INT` , `ENTITY_KEY` `VARCHAR(100)` , `PAYLOAD` `NVARCHAR(MAX)` , `STATUS` `VARCHAR(20)` DEFAULT 'PENDENTE' , `RETRY_COUNT` `SMALLINT` DEFAULT 0 , `LAST_ERROR` `VARCHAR(4000)` , `PRIORITY` `SMALLINT` DEFAULT 0 , `DH_CRIACAO` , `DH_PROCESSAMENTO` , `DH_ALTERACAO` . Índices: `IDX_FCQUEUE_STATUS`(`STATUS` , `PRIORITY` DESC, `DH_CRIACAO`) (:84), `IDX_FCQUEUE_ENTITYKEY_DH` (:628), `IDX_FCQUEUE_ETYPE_STATUS_DH` (:638).

Serviço: `service/QueueService.java` . Enfileiramento em `QueueService.java:80-90` (INSERT com STATUS `PENDENTE`), consumo por `OutboxProcessorJob` que lê `SELECT TOP (?) ... WHERE STATUS='PENDENTE' ORDER BY PRIORITY DESC, DH_CRIACAO ASC` (:150-154) e faz claim para `PROCESSANDO` (:236, com `WHERE IDQUEUE=? AND STATUS='PENDENTE'` — CAS otimista). Dedup de itens em voo por `ENTITY_ID / ENTITY_KEY` (:114-115).

Valores de `ENTITY_TYPE` (`FastchannelConstants:82-87`): `PRODUTO` , `ESTOQUE` , `PRECO` , `PEDIDO_STATUS` , `PARCEIRO` , `TRACKING` .
Valores de `OPERATION` (:100-102): `CREATE` , `UPDATE` , `DELETE` .
Valores de `STATUS` (:91-96): `PENDENTE` , `PROCESSANDO` , `ENVIADO` , `ERRO` , `ERRO_FATAL` , `CANCELADO` . Falha incrementa `RETRY_COUNT` e grava `LAST_ERROR`

(:291-292); sucesso vira `ENVIADO` (:266). Quem alimenta a fila são os listeners (`listener/EstoqueListener` , `PrecoListener` , `ProdutoListener` , `DescontoPromocionalListener` , `NotaFiscalListener` , `TrackingListener`).

4. `AD_FCPEDIDO` — Import de pedido (FC → Sankhya)

DDL: `mssqlserver.sql:123-147` (+ `ALTER` s evolutivos e `ALTER COLUMN NUNOTA INT NULL` em :578). Constraints: `UK_FCPEDIDO_ORDERID` `UNIQUE(ORDER_ID)` ; índices `IDX_FCPEDIDO_NUNOTA` (:157), `IDX_FCPEDIDO_ORDER_STATUS(ORDER_ID, STATUS_IMPORT, NUNOTA)` (:688), `IDX_FCPEDIDO_IMPORT_DH(STATUS_IMPORT, DH_IMPORTACAO)` (:698).

Colunas-chave: `IDPEDIDO` (PK IDENTITY), `ORDER_ID VARCHAR(100)` (id do pedido na FC), `NUNOTA INT NULL` (NUNOTA gerado no Sankhya — **NULL** até o pedido ser criado), `CODPARC` , `STATUS_FC SMALLINT` (código de status FC), `STATUS_SKW VARCHAR(5)` , `STATUS_IMPORT VARCHAR(20)` , `VALOR_TOTAL` , `VALOR_FRETE` , `DH_PEDIDO` , `NOME_CLIENTE` , `CPF_CNPJ` , `ERRO_MSG VARCHAR(1000)` , `DH_IMPORTACAO` , `DH_FATURAMENTO` , `DH_ENTREGA` , `NF_NUMERO` , `NF_SERIE` , `NF_CHAVE` , `TRACKING_CODE` , `OBSERVACAO` .

`STATUS_IMPORT` (constantes `FastchannelConstants:75-78`): `PENDENTE` , `PROCESSANDO` , `SUCCESSO` , `ERRO` . Gravado por `OrderService.upsertOrderMapping()` (`service/OrderService.java:2385` , com fallback JDBC :2432). O `OrderImportJob` importa via `OrderService` (localizar/criar `TGFPAR` , criar nota em `TGFCAB` / `TGFITE`). Migrações de saneamento no DDL corrigem linhas presas em `PROCESSANDO` (V19/V20 — `mssqlserver.sql:1066,1094,1109`): sem `NUNOTA` → volta para `PENDENTE` ; com `NUNOTA` → vira `SUCCESSO` .

5. Tabelas Sankhya nativas utilizadas

Preço — `TGFTAB` , `TGFEXC` e a função `SNK_GET_PRECO`

- **TGFTAB** (cabeçalho de tabela de preço, versionada por `DTVIGOR`): colunas usadas `NUTAB` (PK da versão), `CODTAB` (a "família"/tabela lógica; múltiplas `NUTAB` por `CODTAB`, uma por vigência), `DTVIGOR` , `CODTABORIG` (tabela de origem quando a tabela é derivada por percentual). Colunas custom: `AD_TIPO_FAST CHAR(1)` e `AD_FAST INT` (`mssqlserver.sql:944-954`) usadas no fallback por tipo (`PriceTableResolver.fetchByTipoFast:127`). A "NUTAB vigente" de uma `CODTAB` é sempre `MAX(DTVIGOR)` (padrão em todas as queries de `PriceTableResolver`).
- **TGFEXC** (exceções/preços por produto×NUTAB): checada em `PriceService` (`SELECT TOP 1 1 FROM TGFEXC WHERE CODPROD=? AND NUTAB=?` , :463); índice custom `IDX_TGFEXC_CODPROD_NUTAB` (:668). ⚠ Semântica conhecida (memória Sankhya): `TGFEXC.VLRVENDA` é **percentual** quando `TIPO='P'` e **preço fixo** quando `TIPO='V'` . O addon **não lê VLRVENDA diretamente** — delega o cálculo à função Sankhya para não errar essa semântica.
- **SNK_GET_PRECO(NUTAB, CODPROD, GETDATE())** (função escalar `[sankhya].SNK_GET_PRECO`): é a fonte oficial do preço já calculado. Usada em `PriceResolver.fetchPriceDecimal()` (`service/PriceResolver.java:123` , e fallback JDBC :155) e no SQL de escalonado.

Mapeamento ListPrice vs SalePrice (`PriceResolver.resolve()` :22-35):

- `SalePrice` = `SNK_GET_PRECO(NUTAB alvo, CODPROD)` → preço final (já com ajuste percentual se a tabela for `TIPO=P`).
- `ListPrice` = preço da **tabela de ORIGEM**: se a `NUTAB` alvo tem `TGFTAB.CODTABORIG > 0` , resolve a `NUTAB` de origem (`fetchOriginPrice:47` — `JOIN TGFTAB TORIG ON TORIG.CODTAB = T.CODTABORIG` , `MAX DTVIGOR`) e chama `SNK_GET_PRECO` nela; senão `ListPrice = SalePrice` .
- Ambos convertidos para **centavos** (`toCentavos()` :37 = `movePointRight(2)` arredondado `HALF_UP`), unidade que a API FC espera.

Mapeamento NUTAB ↔ FC PriceTableId — não é hard-coded no Java; vem inteiramente do de-para `AD_FCDEPARA` tipo `TABELA_PRECO` (`COD_SANKHYA` =`NUTAB`, `COD_EXTERNO` =`FC PriceTableId`). O `PriceTableResolver` resolve, para cada FC `PriceTableId`, a **NUTAB vigente** da `CODTAB` do de-para (`findLatestNuTabForFcTable:296` / `resolveTableToNuTabMap:169`), garantindo 1 PUT por tabela FC e evitando `NUTABs` obsoletas. Estado atual em PROD (documentado em `CHANGELOG.md:784-790` , 6 entradas — `NUTAB` do de-para → **NUTAB vigente** → FC `PriceTableId`):

CODTAB	FC PriceTableId	NUTAB vigente
17	4460	4460 (era 4427, 01/04)
19	4459	4459 (era 3060, 2023)
62	4462	4462 (era 4354, 30/01)
64	4461	4461 (era 4442)
65	4458	4458 (era 4443)
66	4457	4457 (era 4444)

Ou seja, os IDs FC 4460/4459/4462/4461/4458/4457 são os `COD_EXTERNO` cadastrados no de-para; os `NUTABs` antigos (4427/3060/4354/4442-4444) ainda existem em `TGFTAB` mas são normalizados para a versão vigente em tempo de query. Caches Guava (TTL 10 min) em `PriceTableResolver` — invalidar via `PriceTableResolver.invalidateCaches()` após escrita em `AD_FCDEPARA` / `TGFTAB` .

Preço escalonado — `TGFDES` (cabeçalho) + `TGFDPQ` (faixas)

Resolvido por `service/PriceBatchResolver.java` (SQL em `BATCH_SQL` , :72-112). Consulta descontos promocionais por quantidade:

- **TGFDES** (cabeçalho de Desconto Promocional): `NUPROMOCAO` , `CODPROD` , `CODTAB` (tabela a que a promoção se aplica — filtro adicionado em 2026-04-24, comentário BUG#1 :54; `CODTAB` `NULL` ou 0 = vale para todas), `NUVERSAO` (usa `MAX(NUVERSAO)` — versão mais recente), `DTINICIAL` , `DTFINAL` (vigência).
- **TGFDPQ** (faixas de quantidade): `QTDE` , `PERCDESC` , `TIPDESC` . **QTDE é o limite SUPERIOR da faixa ("Qtd até")** — não é piso (comentário BUG#2 :63-70). O `MinimumBatchSize` é calculado como `MAX(QTDE anterior)+1` e `MaximumBatchSize = QTDE` .

- **TIPDESC** : 'P' = **percentual** → preço da faixa = `SNK_GET_PRECO(NUTAB,CODPROD) * (1 - PERCDESC/100)` ; caso contrário ('V' = valor) → `PERCDESC` já é o **preço unitário fixo** (`BATCH_SQL:79-84`).

- Faixa-topo aberta ("N e acima"): com 2+ faixas, a maior faixa tem `MaximumBatchSize` estendido para `TOP_TIER_MAX = 999999` (:32, aplicado em :196-208) — fix v1.2.85. Quantidades escritas com escala 0 (`toIntScale:223`) porque a FC grava 0 se receber notação decimal (fix v1.2.85). Preços em centavos.

Promoções expiradas (`DTFINAL < hoje`) são **excluídas** da lista desejada para que o loop de delete no `updatePriceBatches` remova as faixas obsoletas da

FC (:160-172).

- Tabela auxiliar custom `AD_ESCFASTSYNC` (`mssqlserver.sql:1004-1016`): staging/log de sync de escalonado (`NUPROMOCAO`, `QTDE`, `BATCHID`, `PRODUCTID`, `DTALTER`, `MSG`, `STATUS`, `TIPDESC`, `PERCDESC`, `QTDEMIN`, `LISTPRICE`).

Crédito do parceiro — `TGFPAR` + trigger `TRG_INC_TGFCAB` (lado Sankhya)

`TGFPAR` (parceiros) é lido em vários pontos (`OrderService.java:1627,1833,1949,2214,3482` , `PartnerLookupUtil` , `TrackingListener`) para localizar/criar cliente por `CGC_CPF` e obter `COVDEND` preferencial. As colunas `TGFPAR.BLOQUEAR` e `TGFPAR.LIMCRED` (bloqueio e limite de crédito) **não são manipuladas pelo código do addon** — a validação de crédito é executada pela regra/trigger nativa do Sankhya `TRG_INC_TGFCAB` no momento da inclusão da nota (`TGFCAB`). Para o addon, isso é uma dependência externa: se o parceiro estiver bloqueado ou sem limite, a criação da nota falha na camada Sankhya e o pedido cai para `STATUS_IMPORT='ERRO'` em `AD_FCPEDIDO.ERRO_MSG`. (Contexto operacional relacionado: o incidente `CORE_E07763` "Venda Mais" — ver base de conhecimento — foi de registro de base, não do fluxo de crédito deste addon.)

Pedido — `TGFCAB` (cabeçalho) + `TGFITE` (itens)

Destino da importação de pedido (`OrderService`). Colunas custom criadas pelo DDL em `TGFCAB` (`mssqlserver.sql`): `AD_FASTCHANNEL_ID VARCHAR(100)` (:197, + índice filtrado `IDX_TGFCAB_FCID` :207) e `AD_NUMFAST VARCHAR(100)` (:984) — ambas guardam o id do pedido FC e são a base da **idempotência** (checagem de duplicado em `OrderService.isOrderAlreadyImported` :148); `AD_CODVENDEXEC SMALLINT` (:994, vendedor executor); `AD_DESCONTO_FAST DECIMAL(18,2)` (:1124) e `AD_PARCELAS_FAST INT` (:1134). `TGFITE` recebe os itens (CODPROD resolvido pela Regra do SKU acima; `CODLOCAL` com fallback `DEFAULT_CODLOCAL_FALLBACK = 99000000` — `FastchannelConstants`:111).

Resumo de origem dos arquivos

- Config: `model/src/main/java/br/com/bellube/fastchannel/config/FastchannelConfig.java` , `FastchannelConstants.java` , `SourceConfig.java`
- De-Para / SKU: `service/DeparaService.java` , `util/FastchannelProductFilter.java`
- Preço: `service/PriceResolver.java` , `PriceTableResolver.java` , `PriceBatchResolver.java` , `PriceService.java`
- Outbox/fila: `service/QueueService.java` , `job/OutboxProcessorJob.java`
- Pedido: `service/OrderService.java` , `OrderXmlBuilder.java` , `job/OrderImportJob.java`
- DDL: `build/dist/dbscripts/mssqlserver.sql` e `oracle.sql` (empacotados em `WEB-INF/script/`)